



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Instituto de Ciências Exatas e de Informática

Felipe Gabriel de Carvalho
Gustavo Henrique dos Santos Dias
Thiago Henrique Gomes Feliciano

1 SÍNTESE DO ARTIGO

O artigo “A Linear Reduction Model for Parallel Prolog System”, apresenta uma abordagem revolucionária ao desconstruir o backtracking do PROLOG para substituir por outras abordagens mais interessantes. Visto que, o backtracking é o maior gargalo da linguagem, os autores propõem um novo paradigma de execução baseado em Grafos Generalizados AND/OR (GAGs). Nessa nova abordagem, deixa de resolver os problemas como uma árvore e passa a solucioná-los com grafos. Adequando-se ao paralelismo de AND/OR para decidir o melhor caminho para a busca, utilizando os algoritmos de busca baseados em heurística de inteligência artificial como os algoritmos Ao e AO.A* adaptados para operar sobre a estrutura GAG. O algoritmo AO.A*, em particular, demonstra superioridade ao empregar funções de avaliação que combinam custo real acumulado com estimativas heurísticas, mantendo apenas os caminhos mais promissores e eliminando redundâncias computacionais. A implementação prática do modelo materializa-se no Parallel Inference Engine (PIE), uma arquitetura distribuída composta por múltiplos motores de inferência coordenados por um centro de controle unificado. Este sistema explora simultaneamente paralelismo AND e OR, permitindo que diferentes processadores explorem independentemente partes do GAG enquanto mantêm a consistência global das soluções. Em síntese, o artigo oferece uma contribuição significativa ao demonstrar que a combinação estratégica de grafos, busca heurística e processamento paralelo pode superar as deficiências históricas do Prolog. A proposta não apenas elimina o backtracking, mas também estabelece as bases para sistemas de raciocínio lógico escaláveis e eficientes, adequados para as demandas computacionais da Inteligência Artificial contemporânea.

2 DESTAQUES

2.1 Felipe

Em primeiro lugar, um dos destaque do artigo, é ele ser consideravelmente recente para uma linguagem da década de 70 além dele conversar bem com a ideia de IA apresentando propostas oriundas da literatura de inteligência artificial. Os autores apresentam dois algoritmos de busca baseada em heurística, que permitem soluções mais rápidas e menos custosas.

Em segundo lugar, o fruto de pesquisa deles permitiram executar o prolog de forma paralela, aumentando ainda mais a eficiência da linguagem. Nela ainda usam a estratégica lógica das ANDs e ORs, só que com a possibilidade de usarem as duas juntas, ou seja mais um avanço.

Em suma, o artigo traz forte contribuição para a linguagem, visto que trouxe ao menos 3 soluções que melhoram a performance da linguagem, a utilização de grafos para melhor busca tirando a necessidade do uso do backtracking, o paralelismo AND/OR que possibilita a percorrer duas buscas simultâneas, e as propostas dos algoritmos que compõem essa melhoria. Além do artigo trazer exemplos significativos e bem descritos do que ele está explorando.

Figura 1 – Algoritmo A0

Algorithm A0

Step1. Put the initial psg (potential solution graph), which contains only the initial node s , into the open list and calculate $h(s)$ value;

Step 2. Exit and declare search failure if the open list is empty; otherwise continue;

Step 3. Take psg with the smallest h -value in the open list (written as s_0) and put s_0 into the closed list;

Step 4. Exit successfully if s_0 is a solution sub-graph; otherwise continue;

Step 5. Expand s_0 to generate all descendants of s_0 :

$s_1, \dots, s_k$;

Turn to Step 2 if s_0 has no descendant;

Step 6. For each descendant psg s_j , if s_j has been in the open list or the closed list, eliminate s_j from the descendant psg set of s_0 :

(1) Calculate $h(s)$ according to the heuristic function values on the leaf nodes;

(2) If the leaf nodes of some s_j are all terminal nodes, call the solution labeling process for s_j ;

(3) Put s_j into open.

Step 7. Turn to Step 2.

Algorithm A0 has the following main properties: (1) A0 is admissible; (2) A0 is optimal in the sense of expanding psg number.

Fonte: Zhang e Hong (2019)

Estes trechos do artigo, traz dois algoritmos que implementam a melhoria para o grafo GAG, um sendo melhor que outro, por economizar espaço e ter melhor performance. É bem estruturado com passo a passo de como implementar l.

Figura 2 – Algoritmo AO.A*

Algorithm AO.A*:

- Step1. Put the initial tns S_0 into OPEN list, make
 $g(S_0) \leftarrow 0, f(S_0) \leftarrow h(S_0);$
- Step2. Exit and declare failure if OPEN is empty; otherwise continue;
- Step3. Take tns n with the smallest f -value in OPEN, remove it from OPEN, and put n into CLOSED list;
- Step4. Exit successfully if n is the target tns; the solution sub-graph can be found by tracking the rule record stack of tns n ;
- Step 5. Expand n to generate (applying all available rules to n) all descendants tns of n (if n has no descendant, set $f(n) \leftarrow \infty$, and then turn to Step 2), and implement the following calculation on each descendant n_i ;
 $g_i \leftarrow g(n) + c(n, n_i);$
- Step 6. If the descendant tns n_i has not been in OPEN or CLOSED, make
 $g(n_i) \leftarrow g_i, f(n_i) \leftarrow g_i + h(n_i);$
Here $h(n_i) = \sum h(t)$, put n_i into OPEN, and record the rule (n, n_i) by n_i ;
- Step 7. If a descendant n_i has been in OPEN or CLOSED, and $g(n_i) > g_i$, make.
 $g(n_i) \leftarrow g_i, f(n_i) \leftarrow g_i + h(n_i);$
If n_i is in CLOSED originally, move it to OPEN, and replace the original stack with the new rule records of the tns.
- Step 8. Turn to Step 2.

Fonte: Zhang e Hong (2019)

2.2 Gustavo

O artigo "A Linear Reduction Model for Parallel Prolog System" trata da paralelização da linguagem Prolog a partir das estratégias de paralelização OR/AND. Juntamente com isso, é importante ressaltar a alteração do método de busca de árvore para grafos GAG o que permite a implementação de buscas heurísticas como o AO.A* (que seria uma adaptação do algoritmo A* ao modelo de grafos GAG).

O artigo se torna disruptivo no momento em que escolhe o principal "problema" de Prolog, o gargalo de backtracking, como tema de pesquisa e melhoria, demonstrando o forte fundamento teórico dos autores, o que é demonstrado durante todo o artigo com exemplos e partes práticas, o que é um grande ponto positivo.

Outro ponto positivo é a fácil representação e explicação do sistema de PIE (Paralelos Inference Engine), ajudando o leitor a entender completamente o raciocínio que contempla a

paralelização proposta pelo artigo.

Figura 3 – Um Mecanismo de Inferência Prolog Paralelo

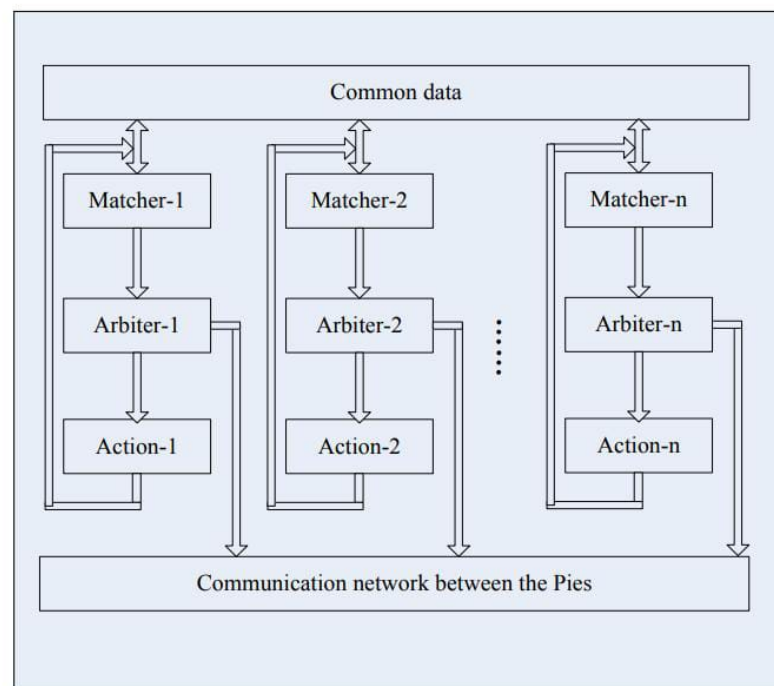


Figure 2. A parallel Prolog inference engine

Fonte: Zhang e Hong (2019)

Com essa foto, os autores conseguem passar para os leitores um entendimento visual do raciocínio, demonstrando a preocupação dos autores com o entendimento do raciocínio proposto, não apenas com a teoria ou com a prática.

2.3 Thiago

O artigo A Linear Reduction Model for Parallel Prolog System, apresenta uma contribuição teórica significativa para o campo da inteligência artificial distribuída e da programação lógica. A pesquisa começa por um problema clássico do Prolog, a ineficiência de seu mecanismo de inferência, fortemente dependente do backtracking e do tratamento implícito das restrições de variáveis. Embora o Prolog seja capaz de expressar raciocínios complexos de maneira natural e declarativa, seu desempenho prático é limitado por operações repetitivas e pouco paralelizáveis.

Os autores retomam a estrutura lógica que sustenta o Prolog, em que cada cláusula de Horn possui a forma geral $AB_1, B_2, \dots, B_n$, significando que o objetivo AAA é verdadeiro se todas as sub-metas B_i também o forem. Essa relação é comumente representada por árvores de derivação AND/OR, nas quais os ramos OR representam alternativas de resolução, e os ramos AND representam a conjunção de condições que precisam ser simultaneamente satisfeitas. Contudo, esse modelo enfrenta limitações importantes: a necessidade de retornar a estados anteriores quando as variáveis compartilhadas não produzem interseções válidas e a execução

essencialmente sequencial das sub-metas.

O artigo mostra o exemplo “Fallible Greek Problem”, em que as metas $\text{fallible}(X)$ e $\text{greek}(X)$ compartilham a variável X . O Prolog tradicional precisa tentar várias atribuições (como $X = \text{Turing}$ e $X = \text{Socrates}$) e, quando uma combinação falha, retorna ao estado anterior para liberar a variável e testar uma nova. Esse processo de retrocesso aumenta exponencialmente o custo de tempo e memória, principalmente yem problemas de larga escala. Para solucionar essas limitações eles propõem um novo modelo de dedução, denominado modelo de redução linear, fundamentado no formalismo dos Generalized AND/OR Graphs (GAG), originalmente introduzido por G. Levi (1976). Diferentemente das árvores tradicionais, o GAG é um grafo direcionado que permite representar explicitamente as restrições (bindings) e as interações entre subproblemas. Essa representação transforma a dedução Prolog em um processo de busca sem retrocesso, em que cada caminho válido no grafo corresponde a uma derivação bem-sucedida. Assim, o problema do backtracking é substituído por um simples processo de verificação de caminhos existentes, eliminando a necessidade de interseções dinâmicas entre variáveis.

A parte que mais interessante no artigo é a utilização dos grafos generalizados (GAG) como solução para as limitações dos modelos tradicionais baseados em árvores. Essa proposta representa um avanço conceitual importante, pois substitui a estrutura hierárquica das árvores de derivação AND/OR por uma representação gráfica mais flexível e expressiva, que consegue mostrar explicitamente as relações e restrições entre as variáveis. As árvores de derivação obrigam o sistema Prolog a percorrer caminhos sequenciais e a recorrer constantemente ao retrocesso (backtracking) para corrigir escolhas inválidas, o uso de grafos permite transformar o raciocínio em um processo de busca sem retrocesso. Essa mudança elimina a necessidade de explorar as subárvores várias vezes, tornando o processo de dedução mais direto, rápido, eficiente e naturalmente paralelizável.

3 CONCLUSÃO

Com isso, chegamos à conclusão de que o projeto era muito ambicioso, pois buscavam solucionar um dos maiores desafios enfrentados pelo Prolog e por outras linguagens lógicas baseadas em inferência, processo de backtracking.

Embora o objetivo principal não tenha sido alcançado, o projeto ajudou mostrando novas alternativas e além de trazer novas ideias para a área. Além disso, o embasamento teórico e as inovações propostas, o artigo mostrou novas ideias para a área de estudo das linguagens lógicas e do Prolog.

Dessa forma, ainda que não tenha obtido o sucesso prático inicialmente almejado, o artigo contribuiu para as pesquisas em linguagens lógicas, promovendo debates, reflexões e além de mostrar o uso de modelos matemáticos antigos na área da informática.

REFERÊNCIAS

ZHANG, W.; HONG, S. A linear reduction model for parallel prolog system. IEEE, Shenyang, China, 2019.